LLM-assisted Path Exploration for RTL Verification

Yan Tan, Xiangchen Meng and Yangdi Lyu†
Microelectronics Thrust, The Hong Kong University of Science and Technology (Guangzhou)
†Corresponding author: yangdilyu@hkust-gz.edu.cn

2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD) | 979-8-3315-3762-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/MLCAD65511.2025.11189172

***Abstract*—Coverage-driven simulation is a widely adopted method for verifying hardware designs. Nevertheless, achieving satisfactory coverage remains a challenge, even with millions of tests applied to modern industrial designs. To tackle this issue, Concolic testing, which combines concrete simulation with symbolic execution, has been introduced to guide simulations through targeted branch mutations. However, traditional path exploration heuristics in Concolic testing are typically manually crafted for specific scenarios, which limits their broader applicability. In this paper, we propose an innovative approach that harnesses large language models (LLMs) to analyze the structure and semantics of Register Transfer Level (RTL) designs. By integrating LLMs into the Concolic testing framework, our method improves path selection by utilizing both static structural features and dynamic simulation data. Recognizing that LLMs are primarily trained for general-purpose tasks, we enhance their effectiveness in hardware verification by constructing a synthetic training corpus and fine-tuning the models accordingly. Our experimental results demonstrate that this approach significantly enhances branch coverage during Concolic testing while incurring negligible overhead.**

***Index Terms*—Large Language Models (LLMs), RTL Verification, Concolic Testing**

I. Introduction

Coverage-driven testing is vital for verifying modern large-scale integrated circuit designs, as it significantly contributes to the identification of bugs and security vulnerabilities in the early stage. When coverage is insufficient, critical flaws can go undetected, leading to substantial financial losses and project delays. As circuit designs grow increasingly complex, achieving high coverage becomes more challenging, even with the use of millions of randomly generated test cases. A key metric for evaluating the effectiveness of verification in Register Transfer Level (RTL) models is branch coverage, which requires that all conditional branches in the RTL code be exercised during simulation, accounting for both the true and false outcomes of each condition [1].

Many directed testing methods have been proposed to improve the coverage of functional verification in the literature. Formal-based test generation approaches, such as model checking and symbolic execution [2], [3], utilize formal methods to generate directed test cases. Despite their effectiveness, these formal approaches often suffer from state explosion problems.

To mitigate the state explosion problem, semi-formal methods like Concolic testing have emerged as effective solutions. Concolic testing [4], [5] combines the efficiency of simulation for large-scale designs with the directness of formal verification to explore hard-to-detect branches, thereby enhancing the overall verification process. While Concolic testing is a promising approach to addressing the state explosion problem, it often encounters path explosion issues as the number of explored cycles increases. Existing Concolic testing frameworks for Register Transfer Level (RTL) models typically rely on expert-designed path exploration heuristics that leverage deep domain knowledge. For instance, Lyu and Mishra [6] proposed a heuristic based on a static edge realignment technique in RTL models to utilize structural information and data flow characteristics for guiding path exploration. Despite their potential, these manually crafted heuristics may not be universally effective across all designs.

To address the limitations associated with path exploration in Concolic testing, we introduce a novel approach that integrates large language models (LLMs) into the Concolic testing framework for RTL verification. Leveraging LLMs' exceptional capability to understand the syntax and semantics of hardware description languages, our method effectively guides path selection to enhance branch coverage and verification efficiency. This integration not only improves the robustness of the verification process but also represents a significant advancement in applying LLM technology to hardware verification tasks.

In our approach, we utilize large pre-trained models via APIs, such as the Generative Pre-trained Transformer (GPT4) [7] by OpenAI, while also fine-tuning smaller LLMs specifically for hardware verification. Recognizing that LLMs are primarily designed for general-purpose applications, we create a synthetic training corpus tailored to the unique requirements of hardware verification. This allows us to fine-tune the models effectively to facilitate local path exploration and optimize overall efficiency. Our methodology employs the Low-Rank Adaptation (LoRA) technique [8] to selectively update a subset of model parameters, enhancing the LLMs' ability to analyze RTL code structures and semantics. *In contrast to traditional Concolic testing, which relies on static and manually crafted heuristics, our approach dynamically mutates branches and explores new paths based on the LLMs' dependency analysis and mutation suggestions.* This innovative strategy allows for more flexible and interpretable path exploration, making our framework a universally applicable solution to the challenges faced in modern RTL verification. The main contributions of our paper are summarized as follows:

1) We proposed the integration of LLMs into the Concolic testing framework for RTL verification. By leveraging the LLMs' abilities to analyze code structure and semantics, our approach enhances the effectiveness of the Concolic testing framework. To the best of our knowledge, this represents the first effort to utilize LLMs to guide path exploration in Concolic testing for RTL models.
2) We constructed a synthetic supervised training corpus that includes problem descriptions and corresponding solutions related to path exploration in Concolic testing. This corpus enables us to fine-tune the LLMs, thereby improving their adaptability and effectiveness for RTL verification tasks.
3) Experimental results demonstrate that our method is robust and adaptable to diverse RTL models, achieving high branch coverage with negligible overhead.

II. Background and Related Work

In this section, we describe the related research on large language models and Concolic testing.

This work was supported by the National Natural Science Foundation of China under Grant #62402412.

A. Large Language Models

Large language models (LLMs) have achieved significant breakthroughs in recent years, demonstrating exceptional capabilities in language understanding and generation [9], [10]. The integration of LLMs into electronic design automation (EDA) tasks has significantly aided engineers in the design and verification of digital systems [11]–[13]. Recent studies [13]–[16] have demonstrated the potential of LLMs in generating Verilog code by fine-tuning models and utilizing advanced prompt engineering techniques. DeLorenzo *et al.* [17] proposed a method that combines LLMs with Monte Carlo Tree Search (MCTS) to produce high-quality RTL code. Cadence also introduced ChipGPT [18] as an LLM-based platform for chip design, which automates HDL code generation to reduce human errors and improve design efficiency. It is also shown that LLMs can be used to generate hardware security assertions to identify and mitigate common vulnerabilities [19], [20]. These studies show the great power of LLMs in understanding hardware description languages.

Although LLMs have proven to be powerful tools, their training stage mainly focuses on general-purpose daily tasks, which can lead to misalignment with specific applications [10], [21]. Moreover, the substantial number of parameters in LLMs makes it prohibitively expensive to train them from scratch for each different use case. Therefore, fine-tuning existing open-source models is a more accessible approach [22]. To improve performance on specific tasks, the LoRA method [8] can be employed to update only a small portion of parameters, enabling LLMs to adapt to downstream tasks.

B. Concolic Testing

Concolic testing is a semi-formal technique for test generation that integrates concrete simulation with symbolic execution. Originally developed for software testing [4], [5], this approach has been successfully adapted for hardware verification [6], [23]. Unlike formal methods that attempt to explore all potential execution paths—often leading to state space explosion—Concolic testing adopts a more targeted approach by iteratively investigating a single execution path and directing it toward a specified target branch. While this strategy alleviates the state explosion issue inherent in formal methods, it introduces the challenge of effective path exploration. To reach a specific target, numerous paths may need to be examined, highlighting the need for well-designed heuristics to optimize coverage and efficiency.

To improve path exploration towards target branches, Lyu and Mishra [6] suggested utilizing structural information and distance metrics obtained from control flow graphs (CFGs) of RTL models. However, the heuristics utilized in existing frameworks are often manually designed based on structural data, which can fall short when addressing hard-to-detect branches in complex scenarios. This paper seeks to overcome these limitations by employing LLM-based methods to guide path exploration, leveraging the analytical strengths of LLMs in hardware description languages.

III. METHODOLOGY

This section presents our LLM-assisted Concolic testing framework designed to improve branch coverage in RTL models through effective path exploration.

A. Overview

The integration of LLMs in a Concolic testing framework is shown in Fig. 1. The framework consists of three main components. The first component preprocesses the RTL model for LLM analysis and Concolic testing. The second component focuses on LLM-assisted path exploration, which aims to identify and prioritize paths that are likely to cover the target branches based on the RTL model and simulation results. The final one utilizes LLM-generated branch mutation suggestions to conduct Concolic testing.

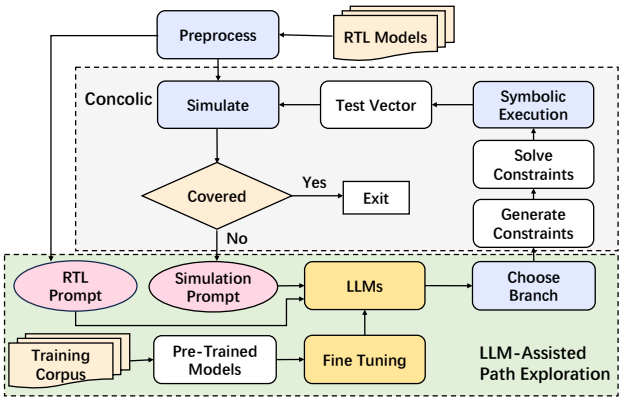

Fig. 1: LLM-Assisted Path Exploration

The process begins with the preprocessing of the RTL model, involving flattening to eliminate hierarchical structure and creating the Design Under Test (DUT). This step also includes instrumenting the RTL model to enable the simulation to gather detailed coverage data and runtime information, providing essential runtime information for LLMs to make decisions.

In the LLM-assisted path exploration stage, the preprocessed RTL model and simulation results are used to generate the RTL system prompt and the simulation prompt, respectively. These prompts give the LLM a comprehensive view of both the static code structure and dynamic simulation traces. Next, the LLM analyzes these prompts to select a group of candidate branch mutations that could potentially increase the likelihood of reaching the target.

In the Concolic testing stage, the chosen branch mutations are processed through symbolic execution to generate new input vectors for subsequent simulations. The simulation results of executing the RTL model with the test vectors from symbolic execution are then fed back into the LLMs to recommend branch mutations in the next round. This iterative process continues until the target is successfully covered or the time budget is reached.

B. Prompts of LLMs

Our path exploration method leverages LLMs to learn from both static features and runtime simulation data. The input prompts for the LLM are composed of two key components: RTL System Prompts and Simulation Question Prompts.

The RTL System Prompts are generated automatically during preprocessing and focus on analyzing static features such as CFG and semantic code information within RTL models. Unlike the manually designed edge realignment heuristic proposed in [6], our method utilizes LLMs to understand the overall structure and logic of the RTL code, laying a robust foundation for further analysis.

The Simulation Question Prompts are automatically generated during the simulation phase of Concolic testing to capture dynamic features. Each prompt includes the initial states, a selection of branch mutations to consider, and a specified output format. We anticipate that the LLM will propose branch mutations based on the provided initial states of all registers. The structured output format guarantees that the LLMs deliver a comprehensive dependency analysis and suggest a prioritized list of branch mutations for the next path

RTL System Prompt ❶

Your task is to **analyse the following RTL code**, extract its **Control Flow Graph**, and **identify the relevant Variables and Conditions**. This analysis will prepare the data necessary for **subsequent path selection**.

```
always @(posedge clk) begin
  case (state)
    2'b00:
      if (in_signal == 2'b01)
        state <= 2'b01; // branch 1
      else
        state <= 2'b10; //branch 2
    2'b01:
      if (in_signal == 2'b10)
        state <= 2'b10; //branch 3
      else
        state <= 2'b00; //branch 4
    2'b10:
      if (in_signal == 2'b01)
        state <= 2'b01; //branch 5
      else
        state <= 2'b11; //branch 6
    default: state <= 2'b00; //branch 7
  endcase
end
```

```
always @(posedge clk)  begin
  if state == 2'b01
    if (in_signal == 2'b10)
      out_signal <= 2'b00;
    else
      out_signal <= 2'b01;
  if state == 2'b11:
    if (in_signal == 2'b11)
      out_signal <= 2'b10; // Target
    else
      out_signal <= 2'b11;
  default: out_signal <= 2'b00;
end
```

Simulation Question Prompt ❷

Trying to **cover target: out_signal <= 2'b10.** Based on the **RTL information** and the provided **Simulation Runtime Data**, determine **which branch should be selected for the next path**.

Initial State:
- state = 2'b00;

Branches To Choose:
- branch 1: state == 2'b00 && in_signal == 2'b01
- branch 2: state == 2'b00 && in_signal != 2'b01
- branch 3: state == 2'b01 && in_signal == 2'b10
- branch 4: state == 2'b01 && in_signal != 2'b10
- branch 5: state == 2'b10 && in_signal == 2'b01
- branch 6: state == 2'b10 && in_signal != 2'b01

Output Format:
- Dependency Analysis :
//Brief explanation of how branches and states interact to achieve the target.
- Proposed Branch Mutations Ranked by Priority:
// List branches in descending order of importance.

Solution ❸

Dependency Analysis :
To reach out_signal = 2'b10, the FSM depends on Branch 2 (in_signal != 2'b01) to transition from state = 2'b00 to state = 2'b10, and Branch 6 (in_signal != 2'b01) to transition from state = 2'b10 to state = 2'b11. Finally, in state = 2'b11, Branch 7 (in_signal == 2'b11) satisfies the dependency out_signal = 2'b10 <= state = 2'b11 ∧ in_signal == 2'b11.

Proposed Branch Mutations Ranked by Priority:
Branch 2: Necessary to transition toward state = 2'b10.
Branch 6: Leads directly to state = 2'b11,
Branch 5: Leads to state = 2'b01, but not critical for target.
Branch 1: Takes the FSM to state = 2'b01, not on target path.

Fig. 2: The Prompts of LLM-assisted Path Exploration

exploration. By integrating both structural and semantic information, these prompts empower LLMs to generate candidate paths that are highly likely to cover the desired target branches.

An example of the prompts used in our LLM-assisted path exploration is shown in Fig. 2. In this scenario, we set the target to be the output signal `out_signal == 2'b10`. The RTL System Prompts provide the LLM with the complete RTL code for necessary preparations in the path exploration process. After simulating the RTL code with certain inputs, we obtain the current state (`state == 2'b00`) and present it to the LLM through the Simulation Question Prompts, which also include a selection of possible branch mutations with their associated conditions. In its output, referred to as the Solution, the LLM provides a detailed analysis of the dependency in the branches and presents a prioritized list of suggested branch

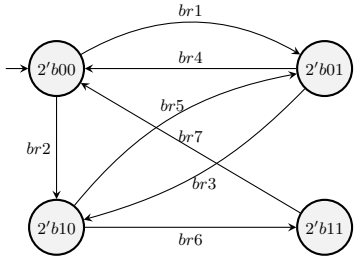

Fig. 3: The finite state machine for the `state` register in Fig. 2. The transitions between states are activated by traversing the branches.

mutations. To validate the correctness of its dependency analysis, we depict the finite state machine (FSM) of the RTL code in Fig. 3 with the starting state set to the initial state provided in the Simulation Question Prompts. By comparing the FSM with the dependency analysis in Fig. 2, we can see that LLMs are capable of conducting a thorough analysis of dependencies within the RTL code. Based on this analysis, the LLM identifies the branch most likely to achieve the target state. In this example, the LLM recommends branch 2 as the best mutation for Concolic testing. While this illustration is straightforward, involving only one register, the LLMs effectively handle more complex analyses, as confirmed by the experiments.

C. LLM-Assisted Path Exploration

The algorithm for LLM-assisted path exploration in Concolic testing is shown in Algorithm 1. The process starts with a random simulation path p. The alternative branches along path p are identified as potential mutation points for exploration in Concolic testing. Subsequently, LLM prompts are generated based on both static structural features and dynamic simulation data, as illustrated in Fig. 2. These prompts are analyzed by the LLMs to identify a list of candidate branch mutations that maximizes the likelihood of reaching the target branch t.

The algorithm then iterates through each proposed branch mutation br according to the rankings provided by the LLM, attempting to symbolically solve a test input that ensures the simulation path traverses this mutation. To solve for a test that passes through a specific branch mutation br, a new constraints stack S is constructed to represent a new path that traverses the same blocks as p before reaching the new branch br and passing through it. If the constraints in stack S are satisfiable, we can obtain a test vector v that results in a simulation path passing through br by solving the constraints with SMT solvers. Assuming that branch br is covered in $br.clock$ cycles, the simulation stage then extends v to N cycles with random inputs and simulates the RTL model to get the new path p. If the target branch t is activated during the simulation, p is considered a successful path. We then add it to the final output and eliminate all the targets covered by p. The algorithm continues until all targets are covered or the iteration limit is reached.

IV. Fine-tuning Large Language Models

Large language models (LLMs) have demonstrated remarkable capabilities in understanding and generating natural languages, as well as several programming languages. However, their general-purpose training can limit their effectiveness in highly specialized tasks such as hardware verification. To address this, fine-tuning serves as an effective strategy to adapt LLMs, extending their pre-trained knowledge with the specific domain knowledge of hardware verification. This section explains selection and implementation of the Low-Rank Adaptation (LoRA) technique for fine-tuning LLMs to improve

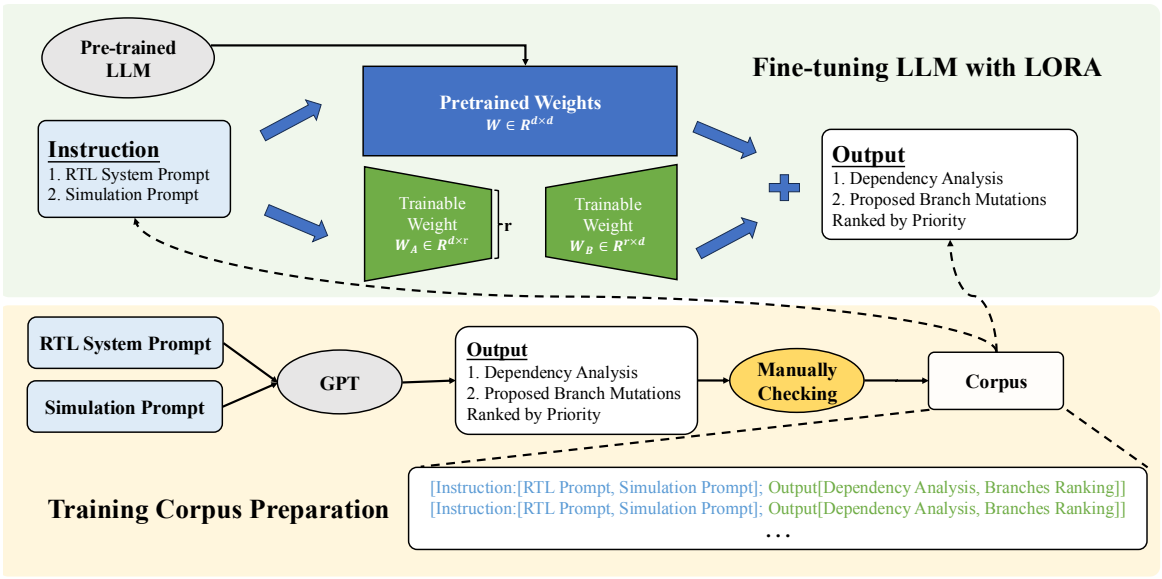

Fig. 4: Fine-tuning Large Language Models with LORA

Algorithm 1 LLM-Assisted Path Exploration Algorithm

Input: RTL model, Search limit L, Unroll Cycle Limit N, Large Language Models LLM, Target list T
Output: Path set $P = \{p_1, p_2, \ldots, p_k\}$
Algorithm:

for target t in T **do** ▷ Cover the current branch target t
  Initialize the random simulation clock for N cycles and obtain the path p
  for L iterations **do**
    ▷ LLM gives suggestions on branch mutation
    Collect the alternative branch set σ of p
    Generate prompts and input into LLM
    LLM suggests a list of branch mutations $B = \{br1, br2, ...\}$ from σ ordered by priority
    ▷ Symbolic Execution on the selected mutation
    for branch mutation br in B **do**
      Build the constraint stack S up to $br.clock$ cycles that follows p and passes through br
      if S is satisfiable **then**
        solve it to get the test vector v
        break
      end if
    end for
    ▷ Simulation with the solved test vector
    Extend v to N cycles with random inputs
    Simulate RTL with v and get the path p
    if the path p activates the target t **then**
      Add p to P
      Remove targets from T that were covered by p
      break ▷ Successfully covered t
    end if
  end for
end for
return Path set $P = \{p_1, p_2, \ldots, p_k\}$

path exploration in RTL verification. In the experimental section (Section V-C), we will demonstrate that fine-tuning significantly improves LLMs' capability to analyze RTL dependencies and provide path exploration suggestions for previously unseen benchmarks.

A. Challenges in Developing an LLM for RTL Verification

Although pre-trained LLMs like GPT have extensive knowledge of natural languages, their understanding of hardware description languages, simulation traces, and hardware-specific semantics is limited. The process of training a new LLM for RTL verification presents several challenges:

- **Resource Constraints:** Training a new and full-scale LLM with billions of parameters requires significant computational resources and memory.
- **Overfitting Risks:** RTL verification tasks often involve relatively small datasets, increasing the likelihood of overfitting during training.
- **Maintaining Generalization**: While training a smaller language model can be quicker and help prevent overfitting, it typically lacks the general reasoning capabilities necessary for handling unseen benchmarks and scenarios.

To address these challenges, we adopted the Low-Rank Adaptation (LoRA) technique [8], which enables efficient fine-tuning the pre-trained LLMs by updating only a small subset of model parameters.

B. Training Corpus Preparation

Effective fine-tuning of LLMs requires a high-quality, domain-specific training corpus, but the process of preparing and constructing such a corpus is complex. To effectively fine-tune the LLM, we constructed a domain-specific training corpus through a process combining automated generation with manual verification, as illustrated in Fig. 4. The corpus generation process begins with running simulations on small RTL benchmarks, providing RTL System Prompts and Simulation Prompts to the most advanced LLM to produce a dependency analysis and a list of proposed branch mutations, which are then manually verified for accuracy and completeness.

1) Selecting RTL Models: While selecting more complex RTL models for fine-tuning could enhance the capabilities of LLMs, we encountered some errors in the dependency analysis generated by even the most advanced vanilla LLMs. In addition, verifying the correctness of the lengthy mutation lists for complex RTL models demands significant manual effort. As a result, our training corpus includes examples from a variety of simpler RTL designs, such as arithmetic logic units (ALUs) and finite state machines, similar to the example shown in Fig. 2. These simpler designs not only reduce the need for extensive manual verification but also help mitigate overfitting and enhance generalization during fine-tuning, given that the number of trainable weights during fine-tuning is significantly lower than in the original LLMs.

2) Input and Output Formats: Considering that our path exploration task in Concolic testing is highly specialized, we constructed a synthetic training corpus that adheres to the format of paired input and output structure depicted in Fig. 2. Each entry in the corpus consists of two key components: an "Instruction Field" and an "Output Field", as shown in Fig. 4. The "Instruction Field" contains the RTL System Prompts and Simulation Prompts, while the "Output Field" contains the dependency analysis and the prioritized list of branch mutations. By fine-tuning on this synthesized training data, the LLM can better interpret complex control-flow logic and variable dependencies within RTL code.

C. Fine-tuning LLM with LoRA

Fig. 4 illustrates the fine-tuning process to train the LLM with the prepared training corpus.

We propose to use the Low-Rank Adaptation (LoRA) technique [8], which enables efficient adaptation of pre-trained models to specific tasks by updating only a small subset of parameters. LoRA introduces low-rank matrices to modify the pre-trained weights during fine-tuning, significantly reducing computational and memory overhead.

- **Original Model Weights**: Let the pre-trained weight matrix be $W \in \mathbb{R}^{d\times d}$, where d is the dimensionality of the input and output vectors.
- **Low-Rank Weights**: LoRA introduces two low-rank matrices, $W_A \in \mathbb{R}^{d\times r}$ and $W_B \in \mathbb{R}^{r\times d}$. Here, d is the size of the original weight matrix and r is the rank hyperparameter in LoRA, representing the intermediate dimension of the low-rank decomposition, where $r \ll d$. The adapted weight matrix W_{merged} is computed as:

$$W_{merged} = W + W_A W_B \tag{1}$$

  This ensures that the model retains its original knowledge while incorporating task-specific adaptations.
- **Training Efficiency**: By freezing the original weights W and training only the low-rank matrices W_A and W_B, LoRA significantly reduces the number of trainable parameters, improving the efficiency of fine-tuning.

Fine-tuning the LLM using the LoRA technique enables the model to leverage its pre-trained knowledge while enhancing its ability in analyzing the RTL code structure and semantics. This allows the model to dynamically guide the path exploration for RTL verification.

V. Experiments

A. Experiment setup

We developed our framework in Python, using Pyverilog [24] for parsing and instrumentation, and Z3 [25] for constraint solving.

To thoroughly evaluate our approach, we conducted experiments using both API-based and locally deployed LLMs.

- The API-based model: GPT-4-turbo [7] by OpenAI, accessed via the official API.
- The local models:
  - PHI-2 [26] with 2.7 billion parameters
  - TinyLlama [27] with 1.1 billion parameters

The configurations for the LLMs were set to top_k=100, top_p=0.9, temperature=0.7, and precision="32-true". The local models were executed utilizing an NVIDIA A30 GPU.

B. Evaluation of LLM-assisted Concolic Testing

To evaluate the effectiveness and efficiency in the RTL branch coverage problem, we compared our proposed method with random simulation and the Concolic framework in [6]. In this experiment, we utilized a range of benchmarks from the OpenCores [28] and ITC'99 [29] datasets.

Table I presents a detailed comparison of branch coverage (Cov) and execution time (s) for the Random, Concolic, GPT-4, PHI-2, and TinyLlama methods across various benchmarks. To ensure consistency and comparability across different models, we excluded the inference time of the LLMs from our runtime measurements, as GPT-4 is accessed via API while PHI-2 and TinyLlama run locally.

From Table I, we can see that GPT-4 consistently achieves the highest branch coverage across all benchmarks. While Concolic testing approach [6] perform well in small benchmarks such as b01, b06, b07, it achieve poor coverage in larger benchmarks such as b12 and b15. When the design becomes more complex, Concolic relies on expert-crafted heuristics cannot effectively guide the hard-to-activate branches exploring. By utilizing the robust code analysis and semantic comprehension abilities of LLMs, we can examine the structural information and integrate it with simulation data to direct branch mutations and improve coverage. Specifically, it enhances coverage by 2.87%, 10.41%, 10.72%, and 5.10% on benchmarks b11, or1200_DCache, or1200_ICache, and or1200_Exception, respectively. For larger benchmarks, we observe more substantial improvements: coverage on b12 increases by 17.86% (from 63.39% to 81.25%) , and on b15 by 40.94% (from 50.25% to 91.19%) compared to the Concolic baseline.

The two smaller LLMs, PHI-2 and TinyLlama, perform well on smaller benchmarks but struggle as benchmark size increases. This is primarily due to two factors: First, PHI-2 and TinyLlama have far fewer parameters—2.7 billion and 1.1 billion, respectively—compared to GPT-4's 175 billion, limiting their ability to capture complex RTL semantics and dependencies. However, fine-tuning could improve their performance for this task, as discussed later. Second, GPT-4 supports context lengths of up to 16,000 tokens, while PHI-2 and TinyLlama are limited to 2,048 tokens, hindering their ability to handle large benchmarks and preserve critical context in long sequences, as noted in Table I.

Overall, our experiments demonstrate the effectiveness of our proposed LLM-assisted path exploration approach in achieving high branch coverage in RTL verification.

C. Evalution of Fine-tuning

In our first experiment, PHI-2 and TinyLlama frequently generated responses that were irrelevant to the given prompts or contained errors, which complicated the identification of correct branch paths and extended the runtime required for effective path exploration. In this experiment, we evaluated the impact of fine-tuning on the performance of our local models, PHI-2 and TinyLlama.We utilized

TABLE I: Comparison of the coverage and time with different methods

Benchmark	Lines	#Br	Random	Concolic [6]		GPT-4		PHI		TinyLlama	
			Cov	Cov	Time (s)	Cov	Time (s)	Cov	Time (s)	Cov	Time (s)
b01	158	26	80.76%	100%	0.01	100%	2.86	100%	44.14	100%	72.08
b06	167	23	82.60%	100%	0.01	100%	4.92	100%	17.13	95.65%	13.616
b07	173	20	80.00%	100%	5.15	100.00%	17.60	70.00%	35.68	70.00%	44.154
b10	263	43	79.06%	100%	0.02	100%	14.94	58.14%	115.73	86.04%	31.314
b11	213	35	74.28%	94.28%	121.67	97.15%	16.11	82.85%	130.03	71.43%	70.392
b12	946	112	56.25%	63.39%	5657.23	81.25%	3287.15	/	/	/	/
b14	1143	195	69.23%	96.41%	133.74	97.94%	108.03	/	/	/	/
b15	1315	193	38.86%	50.25%	5250.81	91.19%	4419.61	/	/	/	/
or1200_DCache	391	48	64.58%	81.25%	101.37	91.66%	124.65	70.83%	247.92	60.41%	297.65
or1200_ICache	190	24	62.50%	89.28%	31.58	100%	46.26	73.07%	164.33	66.67%	144.81
or1200_Exception	587	64	65.62%	91.78%	73.93	96.88%	52.07	79.16%	66.97	76.56%	72.82

Note: / indicates not supported due to context length limitations in PHI-2 and TinyLlama.

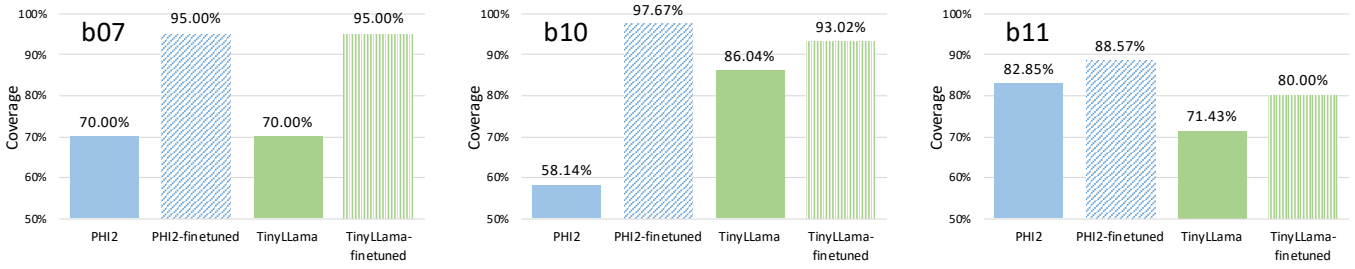

Fig. 5: Comparison of coverage before and after fine-tuning

LoRA to fine-tune the models using a synthetic corpus containing 10,000 entries specifically designed for RTL verification, totaling 49.8 MB in size.

For the fine-tuning process, we utilized PyTorch [30] and litgpt [31] to develop the framework. The training was conducted using the synthetic corpus, with a learning rate of 3×10^{-6}, a batch size of 16, and the Adam optimizer. The models were trained for 10 epochs.The fine-tuning results for PHI-2 and TinyLlama, including costs and validation outcomes, are shown in Table II.

Fig. 5 highlights the improvements in branch coverage post-fine-tuning. For benchmark b07, both models saw coverage rise from 70% to 95%. On b11, TinyLlama's coverage increased from 71.43% to 80%, while PHI-2's rose from 82.85% to 88.57%. In the more complex b10, PHI-2 showed a significant improvement, increasing from 58.14% to 97.67%. These results demonstrate substantial coverage gains through fine-tuning for both LLMs.

TABLE II: Fine-tuning Results

Metric	PHI Fine-tuned	TinyLlama Fine-tuned
Training time (s)	3886.94	3688.16
Training Mem (GB)	20.06	11.16
Validation loss	1.257	0.359
Validation perplexity	3.514	1.431

D. Discussion

The LLM-assisted Concolic testing framework is highly effective in improving path selection and branch coverage, even in medium-sized RTL designs. Results demonstrated that the GPT-4 model outperformed smaller models in achieving high branch coverage across various benchmarks. This can be attributed to its extensive pre-trained knowledge, sophisticated language comprehension, and ability to maintain context in long sequences. Smaller models like TinyLlama performed well on simpler benchmarks but faced challenges with more complex designs, largely due to limitations such as reduced context length and capacity. The PHI model exhibited significant improvements after fine-tuning, demonstrating its stronger inference and generalization capabilities than TinyLlama.

VI. Conclusion

In this paper, we propose an LLM-assisted path exploration framework to improve branch coverage in RTL models. This framework effectively integrates static information from RTL designs with dynamic simulation data in the prompts, allowing LLMs to be highly effective in improving path selection and branch coverage. By employing LoRA, we successfully fine-tuned pre-trained LLMs for the specific task, thereby improving their understanding of hardware description languages and RTL verification. Our approach eliminates the reliance on manual heuristic design in Concolic testing, providing a clear and interpretable path exploration method for RTL verification processes.

References

[1] S. Devadas, A. Ghosh, and K. Keutzer, "An observability-based code coverage metric for functional simulation," *International Conference on Computer Aided Design,International Conference on Computer Aided Design*, Nov 1996.

[2] J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill, and L.-J. Hwang, "Symbolic model checking: 1020 states and beyond," *Information and computation*, vol. 98, no. 2, pp. 142–170, 1992.

[3] A. Kolbi, J. Kukula, and R. Damiano, "Symbolic rtl simulation," in *Proceedings of the 38th Design Automation Conference (IEEE Cat. No. 01CH37232)*. IEEE, 2001, pp. 47–52.

[4] P. Godefroid, N. Klarlund, and K. Sen, "Dart: Directed automated random testing," in *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*, 2005, pp. 213–223.

[5] K. Sen, D. Marinov, and G. Agha, "Cute: A concolic unit testing engine for c," *ACM SIGSOFT Software Engineering Notes*, vol. 30, no. 5, pp. 263–272, 2005.

[6] Y. Lyu and P. Mishra, "Scalable concolic testing of rtl models," *IEEE Transactions on Computers*, vol. 70, no. 7, pp. 979–991, 2020.

[7] OpenAI, "Gpt-3.5-turbo," 2023, accessed: 2023. [Online]. Available: https://platform.openai.com/docs/models/gpt-3-5

[8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models." *ICLR*, vol. 1, no. 2, p. 3, 2022.

[9] K. S. Kalyan, A. Rajasekharan, and S. Sangeetha, "Ammus: A survey of transformer-based pretrained models in natural language processing," *arXiv preprint arXiv:2108.05542*, 2021.

[10] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, vol. 1, no. 2, 2023.

[11] L. Chen, Y. Chen, Z. Chu, W. Fang, T.-Y. Ho, R. Huang, Y. Huang, S. Khan, M. Li, X. Li *et al.*, "The dawn of ai-native eda: Opportunities and challenges of large circuit models," *arXiv preprint arXiv:2403.07257*, 2024.

[12] J. Pan, G. Zhou, C.-C. Chang, I. Jacobson, J. Hu, and Y. Chen, "A survey of research in large language models for electronic design automation," *ACM Transactions on Design Automation of Electronic Systems*, 2025.

[13] J. Gai, H. Chen, Z. Wang, H. Zhou, W. Zhao, N. Lane, and H. Fan, "Exploring code language models for automated hls-based hardware generation: Benchmark, infrastructure and analysis," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, 2025, pp. 988–994.

[14] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, "Chipgpt: How far are we from natural language hardware design," *arXiv preprint arXiv:2305.14019*, 2023.

[15] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Verilogeval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–8.

[16] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, "Openllm-rtl: Open dataset and benchmark for llm-aided design rtl generation," in *Proceedings of 2024 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. ACM, 2024.

[17] M. DeLorenzo, A. B. Chowdhury, V. Gohil, S. Thakur, R. Karri, S. Garg, and J. Rajendran, "Make every move count: Llm-based high-quality rtl code generation using mcts," *arXiv preprint arXiv:2402.03289*, 2024.

[18] Cadence, "Cadence creates industry's first llm technology for chip design," 2023. [Online]. Available: https://community.cadence.com/cadence_blogs_8/b/corporate-news/posts/cadence-creates-industry-s-first-llm-technology-for-chip-design

[19] R. Kande, H. Pearce, B. Tan, B. Dolan-Gavitt, S. Thakur, R. Karri, and J. Rajendran, "(security) assertions by large language models," *IEEE Transactions on Information Forensics and Security*, 2024.

[20] B. Mali, K. Maddala, V. Gupta, S. Reddy, C. Karfa, and R. Karri, "Chiraag: Chatgpt informed rapid and automated assertion generation," in *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2024, pp. 680–683.

[21] Y. Ma, Y. Liu, Y. Yu, Y. Zhang, Y. Jiang, C. Wang, and S. Li, "At which training stage does code data help llms reasoning?" *arXiv preprint arXiv:2309.16298*, 2023.

[22] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652*, 2021.

[23] R. Mukherjee, D. Kroening, and T. Melham, "Hardware verification using software analyzers," in *2015 IEEE Computer Society Annual Symposium on VLSI*. IEEE, 2015, pp. 7–12.

[24] S. Takamaeda-Yamazaki, "Pyverilog: A python-based hardware design processing toolkit for verilog hdl," in *Applied Reconfigurable Computing: 11th International Symposium, ARC 2015, Bochum, Germany, April 13-17, 2015, Proceedings 11*. Springer, 2015, pp. 451–460.

[25] L. De Moura and N. Bjørner, "Z3 Theorem Prover," accessed: 2008. [Online]. Available: https://github.com/Z3Prover/z3

[26] M. Javaheripi, S. Bubeck, M. Abdin, J. Aneja, S. Bubeck, C. C. T. Mendes, W. Chen, A. Del Giorno, R. Eldan, S. Gopi *et al.*, "Phi-2: The surprising power of small language models," *Microsoft Research Blog*, vol. 1, no. 3, p. 3, 2023.

[27] P. Zhang, G. Zeng, T. Wang, and W. Lu, "Tinyllama: An open-source small language model," *arXiv preprint arXiv:2401.02385*, 2024.

[28] "Opencores website," accessed: 2022. [Online]. Available: https://www.opencores.org

[29] F. Corno, M. S. Reorda, and G. Squillero, "Rt-level itc'99 benchmarks and first atpg results," *IEEE Design & Test of computers*, vol. 17, no. 3, pp. 44–53, 2000.

[30] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[31] L. AI, "Litgpt," https://github.com/Lightning-AI/litgpt, 2023.